Black-Box Verification for GNN Ownership via Decision Boundary Fingerprints

Meng Shen, *Member, IEEE*, Luyao Peng, Hao Lu, Yue Qin, Qi Li, *Senior Member, IEEE*, and Liehuang Zhu, *Senior Member, IEEE*

***Abstract*—The widespread adoption and significant development costs associated with Graph Neural Networks (GNNs) increase vulnerability to model stealing attacks, posing a serious threat to intellectual property (IP). Third-party ownership verification provides an effective mechanism to protect IP of model owners while ensuring impartial verification. Current GNN fingerprint verification methods impose impractical requirements, demanding either access to internal model fingerprints (e.g., node embeddings) or detailed information about model stealing attacks. In this paper, we propose Canary, a black-box third-party GNN ownership verification protocol based on fingerprints, where verifiers access only model prediction posteriors and require no knowledge of adversaries' model stealing attacks. Specifically, we generate subgraph inputs as decision boundary fingerprints by maximizing the logits distance between the shadow surrogate and shadow independent model, effectively revealing their posterior probability discrepancies. Experiments across six datasets show that Canary effectively detects ten types of model stealing attacks, achieving a 35.68% higher negative-class F1-score than the state-of-the-art gray-box method Grove while maintaining robustness against seven types of evasion attacks.**

***Index Terms*—Model intellectual property, graph neural networks, model ownership verification.**

I. Introduction

GRAPH Neural Networks (GNNs) demonstrate superior performance across various domains, such as transaction detection [1], [2], decentralized application identification [3] and traffic prediction [4]. This advancement encourages numerous GNN operators to use Machine Learning as a Service (MLaaS) platforms, e.g., Amazon Web Services (AWS) [5] and Microsoft Azure [6], to provide commercial services, where users query the GNNs via APIs and obtain the model output.

Received 13 December 2025; revised 11 April 2026; accepted 15 May 2026. Date of publication 20 May 2026; date of current version 1 September 2026. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB2703800, in part by NSFC Projects under Grant U23A20304, Grant U25A20428, and Grant 62132011, and in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM114. *(Corresponding authors: Meng Shen; Yue Qin.)*

Meng Shen, Luyao Peng, Hao Lu, and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: shenmeng@bit.edu.cn; 3220245311@bit.edu.cn; haolu@bit.edu.cn; liehuangz@bit.edu.cn).

Yue Qin is with the School of Information, Central University of Finance and Economics, Beijing 100081, China (e-mail: qinyue@cufe.edu.cn).

Qi Li is with the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China (e-mail: qli01@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3695130

Since high-quality GNNs generally require significant computational and data resources, *adversaries* can launch model stealing attacks to replicate a *target* GNN model and build their *surrogate* models for cost savings. For example, model extraction attacks [7], [8] leverage the output of the target model for supervised training, and several attacks (e.g. model fine-tuning attacks [9], model retraining attacks [10], and weight pruning attacks [11]) modify the parameter and structure of the target model. These attacks jeopardize the intellectual property (IP) [12], [13], [14] of the target model owner (i.e., the victim).

Third-party ownership verification provides an impartial mechanism to protect model IP, where victims request a third party (e.g., an arbitrator) to verify whether a suspicious model has been stolen from their target models [15]. The primary solution for GNN ownership verification is fingerprint methods [15], [16], [17], [18], which compare the consistency of responses (e.g., gray-box node embeddings [15] or black-box posterior probabilities [16], [17], [18]) of the suspicious and target models without interfering with the training process [19]. Existing ownership verification methods for deep neural networks (DNNs) [20], [21], [22] cannot be directly applied to GNNs. This is because DNN methods, based on sample-independent Euclidean data (e.g., images), optimize node features independently without propagating gradients to neighbors in graph data. It causes nodes to aggregate unsynchronized neighbor information during the forward pass, leading to GNN responses misaligned with the verification goal. Current GNN fingerprint methods impose impractical requirements, demanding that the verifier either has access to internal model fingerprints (e.g., gray-box node embeddings) [15], [18] or employ the same attack as the adversary to extract fingerprints [16], [17]. However, in practical black-box scenarios, the verifier can only access the outputs of both suspicious and target models (e.g., posterior probabilities [16], [23]) and cannot anticipate which attack adversaries may employ.

In this paper, we propose Canary, a black-box verification protocol for GNN ownership based on decision boundary fingerprints. The basic idea is that surrogate models that mimic the target model's functionality, tend to produce decision boundaries more similar to the target model than independent models. The target model and its surrogate will make the same predictions for certain nodes that fall on opposite sides of the decision boundaries of the target and independent models, whereas the predictions from an independent model will differ. These

nodes with their neighborhoods can serve as fingerprints to help the verifier distinguish between surrogate and independent models.

To effectively identify diverse surrogate models based on posterior probabilities, Canary designs three key modules: *fingerprint generation*, *fingerprint validation*, and *ownership verification*. 1) The fingerprint generation module constructs a shadow surrogate model through the weaker black-box model extraction attack [7] (where adversaries only has access to the posterior probabilities of the target model) to simulate the model theft and iteratively optimizes node features to maximize the logit distance between the shadow surrogate model and the shadow independent model, generating subgraph inputs as decision boundary fingerprints to effectively distinguish diverse stronger surrogate models (where adversaries can access the gray-box node embeddings or white-box structures and parameters [7], [9], [10], [11]) from independent models. 2) The fingerprint validation module rejects fingerprints that exhibit posterior probability similarity between the target model and the shadow independent model, preventing malicious victims from claiming that independent models have been stolen from the target model. 3) The third module builds an ownership discriminator that captures the posterior response discrepancies corresponding to fingerprints between the target model and shadow models, allowing these response differences to be transferred to various stronger model stealing attacks and providing verification results.

We extensively evaluate the performance of Canary, particularly demonstrating its effectiveness in identifying four types of model extraction attacks [7], two types of model fine-tuning attacks [9], two types of model retraining attacks [10], and two types of weight pruning attacks [11] across six public datasets (i.e., Citeseer [24], Coauthor [25], Amazon [26], Pubmed [27], DBLP [28], and ACM [29]) and three typical GNN architectures (i.e., GraphSAGE [30], GAT [31] and GIN [32]). Furthermore, the robustness of Canary is demonstrated against seven types of evasion detection methods. Our main contributions are as follows.

- We analyze the decision boundary discrepancies between surrogate and independent models and find that these discrepancies can be used to generate decision boundary fingerprints. The fingerprints of Canary reduce a 4.5% FPR and a 5.33%–16.67% FNR compared to natural fingerprints.
- We propose Canary, a black-box verification protocol for GNN ownership based on decision boundary fingerprints, which effectively distinguishes between surrogate and independent models by posterior probabilities without requiring prior knowledge of the types of model stealing attacks employed by adversaries.
- We comprehensively evaluate Canary on six datasets and three typical GNN architectures against ten types of model stealing attacks. Canary effectively distinguishes between surrogate and independent models, achieving up to 35.68% improvement in the negative-class F1-score [33] over the state-of-the-art gray-box method, Grove [15].
- We demonstrate the robustness of Canary against seven evasion detection methods, where adversaries suffer up to a 34.80% accuracy reduction to evade detection.

II. Background

A. Graph Neural Networks

In this paper, we focus on GNN node classification tasks, which are widely used in transaction detection [1] and decentralized application identification [3]. Since user queries typically involve graphs distinct from those utilized during model training, we consider inductive GNNs [30], which are designed to use different graphs for training and inference.

Node classification GNNs [1], [3] learn a mapping $\mathcal{F}: G \rightarrow Y$ from the graph dataset $D = \{G, Y\}$, where $G = (V, A, X)$, V donates its node set, A denotes its adjacency matrix, and $X \in \mathbb{R}^{|V| \times d}$ represents the node feature matrix where d is the dimension of the feature. $A_{ij} = 1$ denotes an edge between $v_i \in V$ and $v_j \in V$. Each node $v \in V$ has its node feature $x_v \in X$ and an associated label $y_v \in Y$. The node classification process involves aggregating the features of neighboring nodes to derive an embedding h_v of node v. Each layer of a GNN performs the following operations:

$$h_v^{(l)} = \text{UPDATE}\left(h_v^{(l-1)}, \text{AGG}\left(\left\{\text{MSG}(h_u^{(l-1)}) | u \in N(v)\right\}\right)\right) \tag{1}$$

Where $h_v^{(l)}$ is the embedding of node v at layer l, $N(v)$ is the set of neighbors of node v, $\text{MSG}(\cdot)$ is function to construct node passing messages, $\text{AGG}(\cdot)$ aggregates messages from neighboring nodes, and $\text{UPDATE}(\cdot, \cdot)$ integrates this aggregated information with the node's current embedding $h_v^{(l-1)}$ to produce its updated embedding $h_v^{(l)}$, which then used to computate the final posterior probabilities.

B. Model Stealing Attacks on GNNs

Adversaries launch model stealing attacks to mimic the functionality of target models, intending to build cost-effective surrogate models [15], [34]. More specifically, the adversary can carry out three types of model stealing attacks:

Black-box attacks: adversaries carry out black-box *model extraction attacks* [7] by querying the target model to obtain posterior probabilities which are used as supervisory information to train surrogate models. We consider two black-box model extraction attacks: **ProYes**, where queries are made on a complete graph structure, and **ProNo**, where the query graph lacks the adjacency matrix. In the latter, the adjacency matrix is reconstructed using the IDGL framework [35];

Gray-box attacks: adversaries carry out gray-box *model extraction attacks* [7] by querying the target model to obtain node embeddings. Similar to black-box attacks, we consider two gray-box model extraction attacks: **EmbYes** and **EmbNo**;

White-box attacks: adversaries (e.g., industrial espionage) may exploit software and hardware vulnerabilities [36] to acquire model parameters and architecture, enabling them to perform *model fine-tuning attacks* [9], *model retraining attacks* [10], and *weight pruning attacks* [11]. We consider two model fine-tuning attacks: **FTLL**, where the adversary fine-tunes only the last layer, and **FTAL**, where the adversary fine-tunes all layers; two model retraining attacks: **RTLL**, where the adversary re-initializes and retrains only the last layer and **RTAL**, where the adversary re-initializes the last layer and retrains all layers; and two weight pruning attacks: **WP0.3**

and **WP0.9** where the adversary prunes 30% or 90% of the smallest-magnitude weights followed by fine-tuning.

III. Problem Statement

A. GNN Ownership Verification

We focus on the problem of black-box verification of GNN ownership, which involves mainly three participants: the *victim*, the *adversary*, and the third party *verifier*. The victim trains a GNN known as the target model and deploys it to offer commercial services, where the target model outputs posterior probabilities. The adversary can construct surrogate models by launching model stealing attacks [7], [9], [10], [11] on the target model to mimic its functionality. When the victim suspects its target model has been stolen, it provides the verifier with the relevant dataset format, fingerprints, and query access to the target and suspicious model to request verification. The verifier validates fingerprint integrity based on the provided information, and then uses the validated fingerprints to determine whether a suspicious model has been stolen from the target model or a benign model trained independently (i.e., an independent model). The verification results are returned to the victim as evidence for asserting their legal claims.

B. Threat Models

Following the same assumptions as in recent studies on model stealing attacks [7], [8] and ownership verification [15], [37], we assume that adversaries and the verifier can collect public datasets with the same distribution as the training dataset of the target model. We show in Section VI-F that relaxing assumptions about data distribution and scale does not significantly impact the effectiveness of Canary.

The victims may be impersonated by adversaries (i.e., malicious victims) to mislead the verifier into identifying the target model as having stolen a surrogate model [38]. Since surrogate models are released after the target model, existing methods compare model registration timestamps to identify malicious victims [15]. But they are limited to identifying malicious victims who falsely claim earlier-released models as surrogate models. In contrast, we focus on malicious victims who falsely claim later-released independent models of infringement.

The adversaries may possess different knowledge of the target model, respectively, as shown in Section II-B: the posterior probabilities, embeddings, or model parameters and architecture. The posterior probabilities or embeddings are used to perform model extraction attacks [7]. The parameters and architecture are utilized to perform fine-tuning attacks [9], retraining attacks [10], and weight pruning attacks [11].

The verifier is honest but curious [39], which means that it can honestly follow the verification protocol, but is curious about the privacy of the target model. As a result, we assume that the verifier has black-box access only to the target and suspicious models and obtains the posterior probabilities.

C. Design Goals

The design goals of the verification scheme for the ownership of GNNs are described as follows.

- *Effectiveness:* It should effectively distinguish between surrogate models and independent models with low false negative rates and false positive rates.
- *Integrity:* It should prevent victims from falsely claiming that independent models are stolen from their target models.
- *Efficiency:* It should be cost-efficient with a limited number of queries on the target and suspicious model.

IV. The Proposed Canary

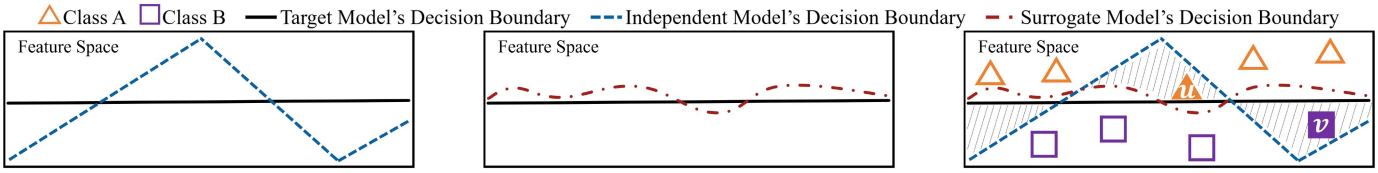
A. Key Insights Into GNN Decision Boundary

Model ownership verification determines whether a suspicious model is a surrogate that replicates the functionality of a target model or an independently trained model. In the following, we examine the external behavior of GNNs that can be utilized to distinguish surrogate from independent models.

In GNN-based node classification, each node is initially assigned a representation (i.e., node features) in the feature space, which serves as the model input. Subsequently, GNN layers iteratively refine these representations by applying feature transformations and aggregating information from neighboring nodes [40]. This process determines the placement and the shape of the decision boundary of the GNN that separates classes in the feature space. Due to randomized initialization of model parameters (e.g., the weight matrix), independently trained GNNs for the same task can place and shape different decision boundaries in the feature space [41], [42], as shown in Fig. 1(a). The decision boundary can be visualized using T-SNE [43]. In contrast, a surrogate model, resulting from model stealing attacks that simulate the behavior and functionality of the target model, tends to produce decision boundaries more similar to those of the target model, as shown in Fig. 1(b). Such discrepancies can be exploited to distinguish surrogate models from independent models.

The key to exploiting these discrepancies lies in the nodes that fall on opposite sides of the decision boundaries of the target model and the independent model, such as the nodes u and v in Fig. 1(c). The target model and its surrogate will make the same predictions for these nodes, while the independent model will differ. In this study, such nodes, along with their neighborhoods, are referred to as *fingerprints*, as they highlight the discrepancy between a surrogate and an independent model. Here, the neighborhood nodes of a central node (e.g., node u) are also considered a part of the fingerprint because they play an important role in updating the representation of the central node through a GNN layer.

To examine the effectiveness and existence of fingerprints, we conducted two case studies using the same experimental settings on six real-world graph datasets (i.e., Citeseer [24], Coauthor [25], Amazon [26], Pubmed [27], DBLP [28], and ACM [29]). Specifically, for each graph dataset, we randomly split it into three subgraphs with an equal number of nodes, retaining the edges within each subgraph. These subgraphs contain 846, 3943, 3543, 3666, 605, and 1530 nodes for Citeseer, PubMed, DBLP, Coauthor, ACM, and Amazon, respectively. We then used the three subgraphs from each dataset to construct a target model, an independent model, and a set of surrogate models, respectively. The target and independent models use



(a) Decision boundaries of the target and independent models.

(b) Decision boundaries of the target and surrogate models.

(c) Decision boundaries of the target, surrogate and independent models.

Fig. 1. An illustration of the decision boundaries of different models in the feature space.

TABLE I
THE DECISION BOUNDARY DEVIATIONS

Datasets	ProYes	FTLL	RTLL	WP0.3	Independent
Citeseer	6.15%	0.24%	2.84%	5.79%	21.39%
Pubmed	6.77%	0.14%	0.10%	0.23%	15.09%
DBLP	12.67%	2.79%	5.28%	3.04%	15.52%
Coauthor	10.01%	1.28%	3.76%	1.53%	10.50%
ACM	5.95%	0.21%	0.33%	0.17%	11.24%
Amazon	8.76%	2.94%	4.51%	1.96%	8.95%

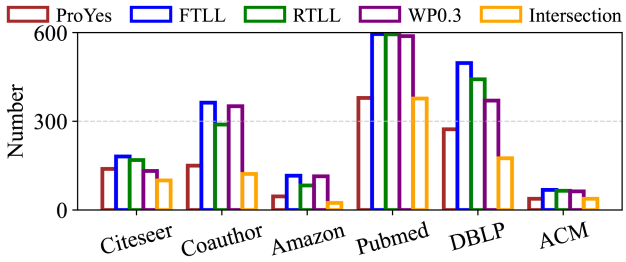


Fig. 2. The number of searched fingerprints in six target model training subgraphs, where a fingerprint means the target model and a surrogate model have the same prediction, while the independent model has a different prediction. The interaction means the number of fingerprints across all surrogate models.

GraphSAGE [30] and the surrogate models are generated by four model stealing attacks (i.e., ProYes [7], FTLL [9], RTLL [10], and WP0.3 [11], as detailed in Section II-B).

Case I aims to demonstrate that decision boundaries of surrogate models are more close to the target model than those of independent models. *Case II* aims to demonstrate that fingerprints are distributed across the real-world datasets.

Case I: We measure decision boundary deviation as the proportion of nodes where surrogate or independent models disagree with the target model in the training subgraph of the target models. Table I shows the decision boundary deviations of the surrogate and independent models on the six target model training subgraphs, demonstrating that the decision boundaries of surrogate models are closer to that of the target model than that of the independent model.

Case II: We search for fingerprints (such as node u in Fig. 1(c)) in the training subgraph of the target models. Fig. 2 shows the number of searched fingerprints in six target model training subgraphs and demonstrates their presence. Moreover, we found that although the number of fingerprints corresponding to surrogate models constructed by the weaker attack ProYes are the least, these fingerprints retain about 90% effectiveness against stronger attacks (e.g., FTLL, RTLL, WP0.3) (as shown in the intersection bar of Fig. 2). This suggests that we can utilize a weaker model stealing attack to extract fingerprints for identifying stronger attacks.

However, the extraction of fingerprints from real-world datasets is highly dependent on and tailored to the specific surrogate models, and requires enumerating all candidate nodes in the datasets. Given the diversity and dynamic nature of model stealing attacks, we aim to design a more efficient verification method to generate fingerprints that are effective in detecting various types of surrogate models.

B. Overview of Canary

Motivated by key insights into the decision boundary of GNNs, we propose Canary, which is a black-box verification method for GNNs using the decision boundary fingerprints. The overview of the Canary is shown in Fig. 3, which consists of three modules.

Fingerprint Generation: This module is performed by the victim, who has white-box access to the target model. The victim trains a shadow surrogate model and a shadow independent model, then generate GNN decision boundary fingerprints by increasing the cosine similarity of the logits between the target and shadow surrogate model, and decreasing the similarity between the target and shadow independent model.

Fingerprint Validation: This module is performed by the verifier, who only has black-box access to the target model and the suspicious model. To prevent false claims from malicious victims, the verifier constructs its own shadow independent model and evaluates the similarity of posterior probabilities of submitted fingerprints between the target model and its shadow independent model.

Ownership Verification: This module is performed by the verifier, who constructs posterior probability distance vectors for fingerprints from both the target and shadow models. These vectors are used to train an ownership discriminator, which captures the posterior response discrepancies corresponding to fingerprints unique to surrogate models and independent models, enabling it to verify the ownership of the suspicious model by matching its posterior probabilities.

V. DESIGN DETAILS OF CANARY

A. Fingerprint Generation

The fingerprint generation module optimizes the subgraphs into decision boundary fingerprints. Specifically, we sample nodes $\{v_1, v_2, \ldots, v_m\}$ as the central node from the training dataset of the target model. For each node v_i, we extend its n-hop subgraph g_i. In this paper, we set $n = 2$, which represents the

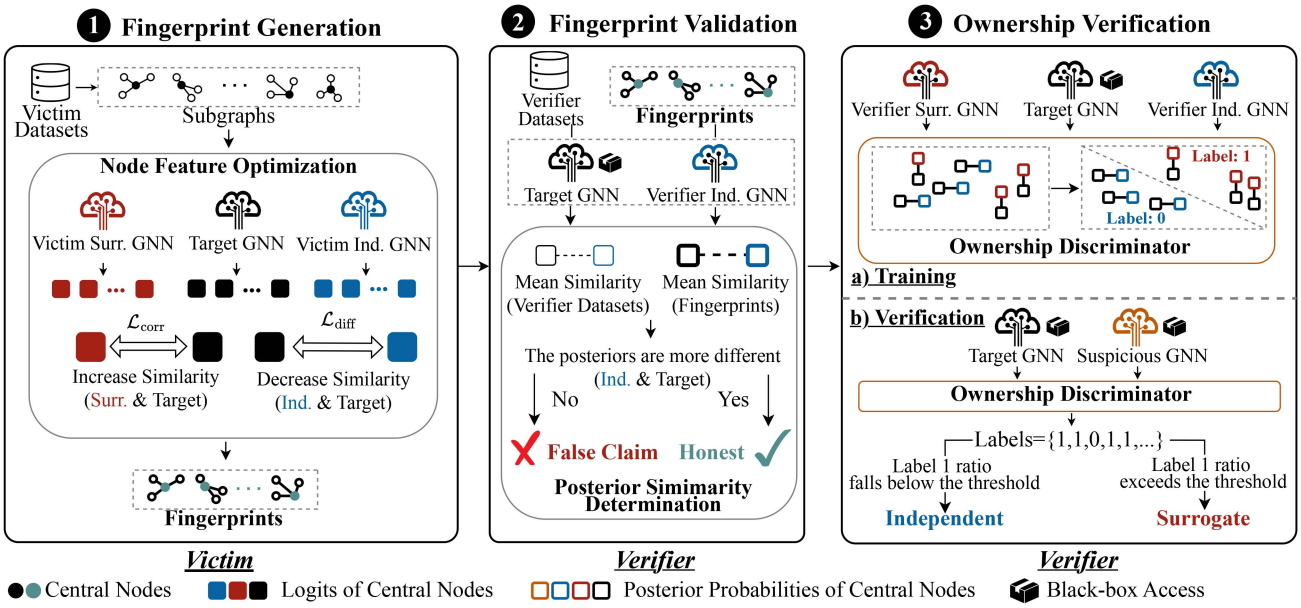


Fig. 3. The overview of Canary.

smallest subgraph that includes both direct and indirect neighbors. To ensure that surrogate models and independent models yield distinct posterior probabilities for its central node, the victim trains shadow models to mimic the decision boundaries of surrogate models and independent models. The victim $\mathcal{O}$ constructs a shadow independent model $\mathcal{F}_{\mathcal{O}}^{I}$ and a shadow surrogate model $\mathcal{F}_{\mathcal{O}}^{S}$ on the dataset $\mathcal{D}^{\mathcal{O}}$. The shadow independent model is independently trained with different initialization parameters than the target model.

To enable Canary to effectively detect surrogate models created by various stronger model extraction attacks, the surrogate shadow model is constructed using a weaker black-box extraction attack (i.e., ProYes, where adversaries can only access the posterior probability of the target model). This is because the surrogate models constructed by the weaker attack exhibit decision boundaries farther from that of the target model than the stronger attacks (e.g., as shown in Table I, where the decision boundary deviations of the black-box ProYes attack exceeds that of white-box attacks FTLL, RTLL, and WP0.3). Consequently, most fingerprints extracted from the weaker surrogate model remain effective against stronger attacks (as shown on the Intersection bars in Fig. 2).

Node Feature Optimization: With the shadow models and sampled subgraphs in place, the victim formulates a loss function $\mathcal{L}_{\text{final}}$ to optimize the features of the subgraph nodes. Given that the logits (the vector before the softmax layer) contain more relative confidence information than the posterior probabilities and the victim has white-box access to the target model, the $\mathcal{L}_{\text{final}}$ increases the logit similarity between the target model and the shadow surrogate model, and decreases this similarity between the target model and the shadow independent model, which consists of three components:

Similarity Loss. It aims to make the target model and the shadow surrogate model produce similar logits on fingerprints. Since predictions depend only on the relative magnitudes (i.e., direction) of logits and logits from different models often exhibit vastly different scales, we employ cosine similarity [44] as the loss function, which measures directional alignment while being invariant to magnitude, thereby better capturing behavioral similarity between models. Given a subgraph g_i, we define the $\mathcal{L}_{\text{corr}}$ to guide the optimization direction:

$$\mathcal{L}_{\text{corr}} = -\text{Cos}^2\langle \mathcal{Z}^T(g_i), \mathcal{Z}_{\mathcal{O}}^S(g_i)\rangle \tag{2}$$

We denote $\mathcal{Z}(g_i)$ as the logit of the central node v_i in g_i, utilizing the cosine similarity squared to maintain the optimization direction's consistency. As $\mathcal{Z}^T(g_i)$ and $\mathcal{Z}_{\mathcal{O}}^S(g_i)$ become similar, $\mathcal{L}_{\text{corr}}$ decreases, ultimately approaching -1.

Difference Loss. The difference loss aims to decrease the cosine similarity of the logits between the target model and the shadow independent model:

$$\mathcal{L}_{\text{diff}} = \text{Cos}^2\langle \mathcal{Z}^T(g_i), \mathcal{Z}_{\mathcal{O}}^I(g_i)\rangle \tag{3}$$

The more deviating $\mathcal{Z}^T(g_i)$ and $\mathcal{Z}_{\mathcal{O}}^I(g_i)$ are, the smaller $\mathcal{L}_{\text{diff}}$ becomes, eventually it converges to 0.

Boundary Loss. Fig. 1 illustrates that nodes near the target model's decision boundary highlight disparities in the decision boundaries of the surrogate models and independent models. Therefore, Canary generates fingerprints near the target model's decision boundary by the optimization term $\mathcal{L}_{\text{bound}}$:

$$\mathcal{L}_{\text{bound}} = \max_{c\in\mathcal{C}}\left(\mathcal{Z}_c^T(g_i)\right) - \text{submax}_{c\in\mathcal{C}}\left(\mathcal{Z}_c^T(g_i)\right) \tag{4}$$

where submax is the second highest value. At the end of this subsection, we provide a theoretical analysis explaining that minimizing the gap between the largest and second-largest logits encourages fingerprints to lie near the decision boundary.

The final loss consists of the aforementioned three parts:

$$\mathcal{L}_{\text{final}} = \mathcal{L}_{\text{corr}} + \lambda_1\mathcal{L}_{\text{diff}} + \lambda_2\mathcal{L}_{\text{bound}} \tag{5}$$

where the difference weight λ_1 and the boundary weight λ_2 are weights of $\mathcal{L}_{\text{corr}}$ and $\mathcal{L}_{\text{diff}}$.

The optimization of subgraphs is guided by $\mathcal{L}_{\text{final}}$. For each central node v_i in each subgraph g_i, we gather the logits from the target and shadow models and then compute $\mathcal{L}_{\text{final}}$. Gradients are

derived through backpropagation of the target model and utilized to optimize features of all nodes in g_i, with l as the learning rate:

$$g_i = g_i - l \cdot \nabla \mathcal{L}_{\text{final}} \tag{6}$$

Considering optimization failures and maintaining the efficacy of fingerprints, the victim selects subgraphs where the shadow surrogate and shadow independent model make different predictions. The final fingerprint set is denoted as:

$$\mathbb{G}_{\mathcal{O}} = \{g_i \mid \mathbb{F}^T(g_i) = \mathbb{F}^S_{\mathcal{O}}(g_i) \wedge \mathbb{F}^T(g_i) \neq \mathbb{F}^I_{\mathcal{O}}(g_i)\}, \tag{7}$$

where $\mathbb{F}(g)$ represents the predicted label of the central node in g. These fingerprints will be submitted to the verifier.

Theoretical Analysis: In a classification model, for a given input g_i, let $c_{max} = \max_{c\in\mathcal{C}}(\mathcal{Z}^T_c(g_i))$ be the predicted class and c_{sub} be the class with the second-highest logit. The decision boundary between these two classes is theoretically characterized by the hyperplane in logit space where $\max_{c\in\mathcal{C}}(\mathcal{Z}^T_c(g_i)) = \text{submax}_{c\in\mathcal{C}}(\mathcal{Z}^T_c(g_i))$. The difference $\mathcal{L}_{\text{bound}} = \max_{c\in\mathcal{C}}(\mathcal{Z}^T_c(g_i)) - \text{submax}_{c\in\mathcal{C}}(\mathcal{Z}^T_c(g_i))$ serves as a measure of the confidence of the target model in classifying the input to class c_{max}. A large $\mathcal{L}_{\text{bound}}$ implies the sample resides deep within the decision region of c_{max}, far from the boundary. Conversely, as $\mathcal{L}_{\text{bound}} \to 0$, the sample approaches the hyperplane between the two classes. Near this hyperplane, the target model typically exhibits distinctive decision behavior that significantly diverges from that of independent models. Thus, minimizing $\mathcal{L}_{\text{bound}}$ explicitly guides the feature optimization to push the generated fingerprints towards the decision boundary.

B. Fingerprint Validation

The fingerprint validation module validates whether the submitted fingerprints originate from malicious victims. We mainly defends against malicious victims who submit carefully crafted false claim fingerprints to mislead the verifier into classifying an independent model as a surrogate model.

Posterior Similarity Determination: Our validation intuition is that, to mislead the verifier, malicious victims should strive to maximizes the logit similarity between the target model and the independent model when generating fingerprint. This similarity should also be reflected in the shadow independent model of the verifier. Therefore, we validate whether the fingerprints are generated for false claims by analyzing their posterior probability performance on the shadow independent model of the verifier. Specifically, the verifier constructs a shadow surrogate model $\mathcal{F}^S_\Omega$ and a shadow independent model $\mathcal{F}^I_\Omega$, and then computes the mean cosine similarity Mean_Ω and $\text{Mean}_{\mathcal{O}}$ between the target model and its shadow independent model for both the verifier's dateset $\mathcal{D}_\Omega$ and the fingerprints $\mathbb{G}_{\mathcal{O}}$. Mean_Ω and $\text{Mean}_{\mathcal{O}}$ can be formulated as:

$$\begin{cases} \text{Mean}_\Omega = \frac{\sum_{g_i\in\mathcal{D}_\Omega} \text{Cos}\left(\mathcal{F}^T(g_i), \mathcal{F}^I_\Omega(g_i)\right)}{|\mathcal{D}_\Omega|} \\ \text{Mean}_{\mathcal{O}} = \frac{\sum_{g_i\in\mathbb{G}_{\mathcal{O}}} \text{Cos}\left(\mathcal{F}^T(g_i), \mathcal{F}^I_\Omega(g_i)\right)}{|\mathbb{G}_{\mathcal{O}}|} \end{cases} \tag{8}$$

Since the malicious victim optimizes the false claim fingerprint to maximize logit similarity between the target model and the independent model, the false claim fingerprint should theoretically exhibit greater posterior probability similarity between the target model and an the shadow independent model than unoptimized data (e.g., the dataset of the verifier). We can identify the false claim fingerprint by validating whether their mean cosine value is smaller than the mean cosine value of the dataset of the verifier, i.e., $\text{Mean}_{\mathcal{O}} < \text{Mean}_\Omega$? Only verified fingerprints can be used for ownership verification.

C. Ownership Verification

The ownership verification module trains an ownership discriminator based on verified fingerprints, and verifies whether the suspicious model has been stolen from the target model.

Training: To capture differences in the responses of surrogate models and independent models to the fingerprint from their posterior probabilities, the verifier obtains posterior probabilities $\mathcal{F}^T(g_i)$, $\mathcal{F}^S_\Omega(g_i)$, and $\mathcal{F}^I_\Omega(g_i)$ and computes six distance metrics between $\mathcal{F}^T(g_i)$ and $\mathcal{F}^S_\Omega(g_i)$ to concatenate a distance vector labeled as 1 (indicating positive samples), while the distance metrics between $\mathcal{F}^T(g_i)$ and $\mathcal{F}^I_\Omega(g_i)$ are concatenated to form negative samples with label 0. The training dataset $\mathbb{D}_\Omega$ of the discriminator consists of these samples:

$$\mathbb{D}_\Omega = \begin{pmatrix} \left\{\text{Distance}\left(\mathcal{F}^T(g_0), \mathcal{F}^I_\Omega(g_0)\right), 0\right\}, \\ \left\{\text{Distance}\left(\mathcal{F}^T(g_0), \mathcal{F}^S_\Omega(g_0)\right), 1\right\}, \\ \cdots \\ \left\{\text{Distance}\left(\mathcal{F}^T(g_m), \mathcal{F}^I_\Omega(g_m)\right), 0\right\}, \\ \left\{\text{Distance}\left(\mathcal{F}^T(g_m), \mathcal{F}^S_\Omega(g_m)\right), 1\right\}, \end{pmatrix} \tag{9}$$

Where m is the number of fingerprints in $\mathbb{G}_\Omega$. Given two posterior probabilities p_1 and p_2, the Distance$(\cdot,\cdot)$ function computes distance vectors containing six dimensions:

Cosine Similarity [44] measures the direction similarity between two vectors, defined as:

$$\text{Cos}(p_1, p_2) = \frac{p_1 \cdot p_2}{|p_1| \cdot |p_2|} \tag{10}$$

Pearson Correlation Distance [45] measures the linear dependence:

$$\rho(p_1, p_2) = \frac{\text{cov}(p_1, p_2)}{\sigma(p_1) \cdot \sigma(p_2)} \tag{11}$$

Chebyshev Distance [46] is a ℓ_∞ metric and measures the biggest deviation between two vectors:

$$\ell_\infty(p_1, p_2) = ||p_1 - p_2||_\infty \tag{12}$$

Euclidean Distance [47] measures straight-line distance:

$$\ell_2(p_1, p_2) = ||p_1 - p_2||_2 \tag{13}$$

Kullback-Leibler Divergence [48] measures the discrepancy between two probability distributions. We use posterior probabilities as input:

$$KL(p_1, p_2) = \sum p_1 \log \frac{p_1}{p_2} \tag{14}$$

Jensen-Shannon Divergence [49] is a variation of KL and solves its asymmetric problem:

$$JS(p_1, p_2) = \frac{1}{2} KL\left(p_1, \frac{p_1 + p_2}{2}\right) + \frac{1}{2} KL\left(p_2, \frac{p_1 + p_2}{2}\right) \tag{15}$$

With the dataset $\mathbb{D}_\Omega$, the verifier employs a multilayer perceptron (MLP) as a binary discriminator $\mathcal{B}$, comprising three linear layers, to learn the decision boundary knowledge of the target model, the shadow surrogate model, and the shadow independent model to verify unseen suspicious models.

Verification: The verifier utilizes the fingerprints $\mathbb{G}_\mathcal{O}$ of the victim and a skilled ownership discriminator $\mathcal{B}$ to determine whether the suspicious model $\mathcal{F}^?$ has been stolen from the target model. Specifically, for the central node v_i of each fingerprint g_i, the verifier queries the posterior probabilities $\mathcal{F}^?(g_i)$ of the suspicious model. For each pair of posterior probabilities $\mathcal{F}^T(g_i)$ and $\mathcal{F}^?(g_i)$, the verifier uses the distance function Distance$(\cdot,\cdot)$ to compute six distance metrics and concatenates them to construct the verification dataset $\mathbb{D}'_\Omega$. Subsequently, the discriminator infers a set of binary predictions $Y = y_1, y_2, \ldots, y_n$ on $\mathbb{D}'_\Omega$. $y = 1$ suggests that the posterior probabilities of the corresponding fingerprints in the suspicious and target models are similar, thus endorsing the hypothesis that the suspicious model may be a surrogate.

To ensure the effectiveness of model ownership verification, we comprehensively consider the judgment results of the entire fingerprint set $\mathbb{G}_\mathcal{O}$. We define the proportion of fingerprints for which the discriminator predicts 1 as the Fingerprint Matching Rate (FMR), serving as a basis for verification:

$$\text{FMR} = \Sigma_{d \in \mathbb{D}'_\Omega} \mathcal{B}(d) / |\mathbb{D}'_\Omega| \tag{16}$$

Where FMR $\in [0, 1]$. Ideally, if the suspicious model is surrogate model, the FMR should be as close as possible to 1. We then compare the FMR to a threshold τ that is a fixed 50% threshold based on majority voting. If FMR $> \tau$, the suspicious model will be considered as a surrogate model.

VI. Evaluation of Canary

In this section, we comprehensively evaluate the performance of Canary in various settings on a machine equipped with an i7-13700 K CPU and RTX 3080 GPU. Particularly, we would like to study the following questions:

- *Q1:* Can Canary effectively distinguish between surrogate models and independent models with low FNR and FPR?
- *Q2:* Can Canary prevent false claims by malicious victims?
- *Q3:* Can Canary be cost-efficient with a limited number of queries on the target model and suspicious models?
- *Q4:* Can Canary effectively defend against evasion?
- *Q5:* How sensitive is Canary to variations in loss functions, distance metrics, data assumptions, hyperparameters, and the shadow surrogate model types?

A. Experiment Setting

Dataset: We evaluate Canary across six graph datasets as shown in Table II. Citeseer [24], Pubmed [27], and DBLP [28] are citation networks. Coauthor [25] and ACM [29] are co-authorship networks, and Amazon [26] is the co-purchase network of photos.

TABLE II
Details of Datasets

	Citeseer	Pubmed	DBLP	Coauthor	ACM	Amazon
Nodes	4230	19717	17716	18333	3025	7650
Edges	10674	88648	105734	163788	26256	238162

We randomly divide each dataset into five parts (i.e., $\mathcal{D}^T$, $\mathcal{D}^\mathcal{O}$, $\mathcal{D}^S$ ($\mathcal{D}^I$), $\mathcal{D}^\Omega$ and $\mathcal{D}^V$), each comprising 20% of the total dataset. The dataset $\mathcal{D}^T$ is used to train the target model and generate the fingerprints of the victim. The dataset $\mathcal{D}^\mathcal{O}$ is used to train shadow models of the victim $\mathcal{O}$. The dataset $\mathcal{D}^\Omega$ is used by the verifier Ω to verify model ownership. The dataset $\mathcal{D}^\mathcal{S}$ ($\mathcal{D}^I$) is used to train surrogate models of adversaries and independent models. The dataset $\mathcal{D}^V$ is used for model testing.

Metrics: We mainly evaluate three metrics while considering surrogate models as positive samples and independent models as negative samples: False Positive Rate (FPR), False Negative Rate (FNR), and Negative F1-score (F1-neg).

- *False Positive Rate (FPR)* [15] measures the proportion of independent models incorrectly classified as surrogate models. It can be computed as FPR $= \frac{FP}{TN+FP}$.
- *False Negative Rate (FNR)* [15] measures the proportion of surrogate models incorrectly classified as independent models. It can be computed as FNR $= \frac{FN}{TP+FN}$.
- *Negative F1-score (F1-neg)* [33] is the harmonic mean of precision and recall for the independent models whose number is less than that of surrogate models in this section. It can be computed as $\text{F1-neg} = \frac{2 \times \text{Precision}_{\text{neg}} \times \text{Recall}_{\text{neg}}}{\text{Precision}_{\text{neg}} + \text{Recall}_{\text{neg}}}$ where $\text{Precision}_{\text{neg}} = \frac{TN}{TN+FN}$ and $\text{Recall}_{\text{neg}} = \frac{TN}{TN+FP}$.

Model Architectures: We use three architectures widely evaluated in stealing attack [7] and ownership verification [15], i.e., GraphSAGE [30], GAT [31], and GIN [32]. We discuss the feasibility of Canary on other architectures in Section VII.

- *GraphSAGE:* We use a three-layer GraphSAGE model consisting of two aggregation layers and a linear layer mapping the classification output. The aggregator uses the MEAN method and the dropout is set to 0.5 to prevent overfitting. The neighborhood sample sizes are set to 25 and 10 for batch training, respectively.
- *GAT:* We employ a three-layer GAT model, maintaining a fixed neighborhood sample size of 10 per layer. The first two layers consist of four attention heads each, while the final layer is dedicated to classification.
- *GIN:* We use a three-layer GIN model with a fixed neighborhood sample size of 10 at each layer.
- *Extraction Attack Models:* For the architecture of the surrogate model for model extraction attacks, we use two aggregation layers as the victim model and two linear layers. The hidden layer sizes are 256, 256 and 100, respectively.

For the architectures of the target model and independent models, the sizes of their hidden layers are 256 and 64. The embeddings are derived from the last hidden layer, whose size is 64. For each dataset and model architecture, we train one target model and 100 independent models by random seeds.

TABLE III
THE AVERAGE MODEL FIDELITY OF DIFFERENT SURROGATE MODELS

Model	Dataset	Surrogate Model (%)									
		ProYes	EmbYes	ProNo	EmbNo	FTLL	FTAL	RTLL	RTAL	WP0.3	WP0.9
GraphSAGE	Citeseer	86.81	87.70	84.15	85.86	85.07	86.24	82.46	84.08	87.57	85.61
	Pubmed	94.54	94.20	91.63	92.37	96.70	91.41	94.66	89.91	92.04	90.75
	DBLP	86.55	82.47	88.13	81.40	94.49	92.60	90.81	91.68	93.59	92.45
	Coauthor	84.01	90.31	83.93	90.27	93.67	93.32	79.16	92.41	93.60	94.00
	ACM	94.86	94.41	92.00	92.44	97.96	97.16	94.04	95.76	95.77	96.87
	Amazon	93.97	86.41	83.63	86.20	89.75	90.92	54.28	78.04	91.11	87.98
GAT	Citeseer	88.12	82.82	81.76	78.37	90.03	87.73	88.15	82.96	81.81	77.14
	Pubmed	90.82	94.46	81.82	91.33	96.09	93.13	95.86	92.44	21.32	39.42
	DBLP	93.39	89.09	93.28	89.65	96.57	92.33	96.60	92.70	79.76	88.18
	Coauthor	81.97	85.33	83.11	85.45	95.52	93.42	95.63	93.44	6.18	5.18
	ACM	92.50	92.57	92.86	93.01	98.14	92.96	97.25	92.76	63.61	91.52
	Amazon	84.36	86.25	82.17	78.41	93.35	93.25	87.20	89.19	14.78	17.88
GIN	Citeseer	71.40	82.66	79.95	87.56	93.02	88.07	94.67	88.62	88.69	67.11
	Pubmed	86.98	91.22	86.40	91.56	95.92	92.65	96.84	93.22	69.68	59.18
	DBLP	73.34	83.85	73.02	83.43	96.49	91.85	90.61	81.36	87.19	67.06
	Coauthor	73.22	76.69	70.12	76.40	86.22	75.74	88.37	80.29	23.77	20.17
	ACM	85.42	87.79	85.19	87.14	89.16	84.91	92.79	85.65	63.49	45.55
	Amazon	72.61	75.32	76.27	79.60	89.48	84.78	86.62	79.80	55.71	52.16

Model Stealing Attacks: Adversaries can launch ten different model stealing attacks mentioned in Section II-B. For each target model, we train 100 surrogate models of each attacks with different random seeds. The fidelity $\left(\text{Fidelity} = \frac{\#\text{surrogate model's predictions} = \text{target model's predictions}}{\#\text{all predictions}}\right)$ of surrogate models is shown in Table III. In this paper, we build shadow surrogate models using only ProYes to identify these ten attack types.

Methods to Compare: We compare with four model ownership verification methods, i.e., *Random Graph (RG) [19]*, *Pairs* [22], *Grove* [15], and *OwnVer* [16]. **Random Graph (RG)** is a watermarking method which embeds a random graph as a watermark into the target model. The model with high-accuracy predictions on the random graph will be considered a surrogate model. **Pairs** is a black-box fingerprinting DNN method verified by the victim. We applied it directly to GNN by generating subgraph pairs with the same label on the target model. The model that achieves high accuracy in precisely predicting these subgraph pairs will be considered a surrogate model. **Grove** is a gray-box fingerprinting method verified by a third party. It develops a embedding similarity discriminator. A model that exhibits significant similarities to the target model will be considered a surrogate model. **OwnVer** is a black-box fingerprinting method verified by a third party. It masks the target model data and constructs a posterior probability similarity discriminator to identify surrogate models.

To ensure a fair comparison, all compared methods that require shadow surrogate models are implemented under the same restricted setting: each target model is paired with one shadow surrogate model constructed using ProYes and one shadow independent model. This setting is weaker than the original configurations recommended by **Grove** (3 shadow independent and 2 shadow surrogate models) and **OwnVer** (40 shadow independent models and 40 shadow surrogate models).

B. Evaluation on Effectiveness

In this subsection, we evaluate the FNR, the FPR, and the F1-neg score of Canary in response to Q1. The results shown in Table IV demonstrate that Canary is effective in distinguishing between surrogate and independent models. Compared to the SOTA gray-box method, Grove [15], Canary can maintain both FNR and FPR close to 0, while achieving a 0.78%–35.68% higher F1-neg score.

FNR: As shown in Table IV, Canary achieves a near-zero FNR in most model stealing attacks. The FNR of Canary on EmbYes is 0, while the ProYes increases FNR by 1.33%–17.00% relative to EmbYes. This is because EmbYes uses more informative node embeddings for theft, resulting in decision boundaries that are similar to those of the target model. The reconstructed graph structure from ProNo and EmbNo incurs errors, leading to a greater difference in decision boundaries between surrogate models and the target model, which increases the FNR. Compared with other datasets, Canary increases a 33.33% higher FNR for WP0.3/WP0.9. It stems from failed model stealing, as indicated by the low fidelity 6.18%/5.18% of surrogate models in Table III. Canary achieves a 1%–5.33% higher FNR than the gray-box fingerprinting method, Grove [15], on other fine-tuning attacks, retraining attacks, and pruning attacks.

Grove [15] obtains embeddings in a gray-box manner and achieves ideal FNR in most attacks, but its overall FNR is still higher than Canary. Apart from the misjudgment caused by the stealing failure, Grove also achieves a 3.00%–100% higher FNR than Canary for FTAL and RTAL on most datasets. These attack methods increase the difference in node embeddings by adjusting all model parameters, thereby reducing the effectiveness of Grove based on node embedding similarity.

OwnVer [16] also exhibits a higher FNR than Canary under model fine-tuning, retraining, and pruning attacks on the Coauthor, ACM, and Amazon datasets. This is because Grove [15] and OwnVer [16] did not use these attacks to build shadow surrogate

TABLE IV
THE EFFECTIVENESS OF CANARY IN DISTINGUISHING BETWEEN SURROGATE MODELS AND INDEPENDENT MODELS

Datasets	Methods	FPR (%)	FNR (%)										F1-neg (%)
			Extraction				Fine-tuning		Retraining		Pruning		
			ProYes	EmbYes	ProNo	EmbNo	FTLL	FTAL	RTLL	RTAL	WP0.3	WP0.9	
Citeseer	RG [19]	0.00	91.33	95.33	96.67	89.67	33.33	33.33	62.00	50.00	63.33	57.00	47.17
	Pair [22]	97.93	0.00	0.00	0.33	0.00	0.00	1.67	0.00	0.00	0.00	0.00	4.03
	Grove [15]	6.00	24.67	0.00	17.00	0.00	0.00	5.33	0.00	23.33	33.33	33.33	78.44
	OwnVer [16]	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	–
	Canary	**0.00**	**0.00**	**0.00**	15.67	11.33	**0.00**	5.33	**0.00**	4.33	**0.00**	**0.00**	**94.24**
Pubmed	RG [19]	0.00	66.67	64.00	67.67	70.33	15.00	29.33	43.33	59.00	66.67	66.67	52.23
	Pair [22]	38.44	0.00	11.00	26.00	27.00	67.33	0.00	0.00	0.00	0.00	0.00	59.96
	Grove [15]	22.22	0.00	0.00	0.00	0.00	0.00	3.00	0.00	35.00	33.33	62.33	69.97
	OwnVer [16]	66.67	0.00	0.00	0.00	0.00	33.33	0.00	0.00	0.00	0.00	0.00	46.15
	Canary	18.67	1.33	**0.00**	3.00	**0.00**	**0.00**	**0.00**	1.00	**0.00**	**0.00**	**0.00**	**88.83**
DBLP	RG [19]	0.00	94.33	100.00	100.00	100.00	66.67	66.67	66.67	100.00	100.00	100.00	40.15
	Pair [22]	40.33	66.67	66.67	66.67	64.67	0.00	0.00	0.00	19.67	16.33	2.33	45.78
	Grove [15]	8.56	22.67	24.00	7.33	5.00	0.00	58.67	0.00	59.67	86.00	100.00	58.51
	OwnVer [16]	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	–
	Canary	**0.00**	17.00	**0.00**	18.67	**0.00**	**0.00**	**0.00**	**0.00**	1.33	**0.00**	**0.00**	**94.19**
Coauthor	RG [19]	0.00	100.00	100.00	100.00	100.00	66.67	66.67	66.67	66.67	66.67	66.67	40.00
	Pair [22]	87.78	32.00	1.67	31.33	0.33	0.00	0.00	0.00	0.00	0.00	0.00	18.24
	Grove [15]	8.56	0.00	0.00	0.00	0.00	0.00	43.67	0.00	100.00	88.00	79.00	61.99
	OwnVer [16]	0.00	0.00	0.00	0.00	0.00	33.33	66.67	0.00	66.67	33.33	33.33	72.00
	Canary	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	33.33	33.33	**90.00**
ACM	RG [19]	0.00	72.00	73.33	86.33	97.33	21.33	32.67	41.33	33.33	33.33	64.33	51.93
	Pair [22]	99.56	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.33	0.00	0.00	0.88
	Grove [15]	23.55	0.00	0.00	0.00	0.00	33.33	5.33	3.33	40.67	0.00	27.67	71.71
	OwnVer [16]	100.00	0.00	0.00	0.00	0.00	66.67	66.67	5.00	33.33	21.00	0.00	–
	Canary	17.11	5.00	**0.00**	10.67	2.00	**0.00**	1.00	**0.00**	**0.00**	3.00	**0.00**	**87.20**
Amazon	RG [19]	0.00	100.00	100.00	100.00	100.00	33.33	45.33	33.33	60.67	63.67	66.67	46.05
	Pair [22]	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	3.00	0.00	–
	Grove [15]	22.56	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	4.00	86.63
	OwnVer [16]	28.11	0.00	0.00	0.00	0.00	100.00	99.33	0.00	0.00	33.33	66.67	52.93
	Canary	19.78	1.67	**0.00**	5.00	**0.00**	**0.00**	**0.00**	**0.00**	3.33	**0.00**	**0.00**	**87.41**

models when training the discriminator in the settings of this paper.

RG [19] is ineffective at identifying surrogate models across various attacks. The reason lies in the objective of model extraction attacks to replicate the functionalities of the target model, which do not capture hidden watermarks unrelated to the main classification tasks. Furthermore, fine-tuning, retraining, and pruning attacks eliminate the watermark within the target model, thereby increasing the FNR for RG.

It is noteworthy that although Pair [22] exhibits a near-zero FNR across all attack methods, it demonstrates an FPR close to 100% when distinguishing independent models. This is because the tasks of the independent models and the surrogate models are identical, and most of the fingerprints of Pair yield the same outputs in both models, leading to misclassification of the independent models.

FPR: Canary achieves the lowest FPR among all fingerprint methods. This is because Canary maximizes the logit distance between the target model and the shadow independent model during the generation process. Consequently, the fingerprints can create significant differences in the posterior probabilities of independent models and the target model, thereby facilitating the identification of independent models by Canary.

Grove [15] compares the similarity of node embeddings. Since independent models and the target model have the same node classification task, they can still produce similar node embeddings for some input samples, which leads to an increase in the FPR of independent model detection.

OwnVer [16] achieves a higher FPR than Canary due to its assumption that the target model and independent models are trained under the same graph structure, which differs from the more realistic setting in Canary, where the training data for the target and independent models are disjoint.

Pair [22] exhibits a high FPR when distinguishing independent models because its fingerprint pairs generate identical outputs in independent models, leading to misjudgments.

RG [19] achieves 0% FPR in all datasets because the independent models do not contain the watermark, which helps RG to effectively identify independent models. But it cannot effectively distinguish surrogate models.

F1-neg: Considering both FNR and FPR, Canary achieves a 0.78%–35.68% higher F1-neg than other methods across all datasets. The superiority of Canary is attributed to its ability to generate fingerprints that reflect the different decision boundaries of independent models and surrogate models. Compared to Grove and OwnVer, which uses raw data as input, and Pair, which generates random fingerprint pairs, Canary's fingerprints create a greater disparity in model outputs between the surrogate models and independent models, thus improving Canary's ability to differentiate them. When FPR = 100, it results in the denominator of F1-negative being 0, making it impossible to compute and we represent it by "–".

C. Evaluation on Integrity

In this subsection, we evaluate the integrity of Canary in response to Q2, i.e., the success detection rate (SDR = $\frac{\#\text{detected fingerprint sets}}{\#\text{fingerprint sets}}$) of Canary against the false claim fingerprint sets from malicious victims. Specifically, we iteratively optimize

TABLE V
THE SUCCESS DETECTION RATE (SDR) OF CANARY FOR FALSE CLAIM FINGERPRINT SETS

Model	SDR (%)		
	GraphSAGE	GAT	GIN
Citeseer	100.00	100.00	100.00
Pubmed	100.00	100.00	100.00
DBLP	100.00	100.00	100.00
Coauthor	100.00	100.00	100.00
ACM	100.00	100.00	100.00
Amazon	100.00	100.00	100.00

node features by transforming $\mathcal{L}_{\text{final}}$ into $\mathcal{L}_{\text{final}} = -\mathcal{L}_{\text{diff}}$ to generate fingerprints that have the same predictions on the target model and the shadow independent model of the victim. We randomly generate 100 sets of false claim fingerprints across six datasets and three model architectures, and record the SDR of the fingerprint validation module in Canary.

As shown in Table V, Canary achieves a 100% SDR for false claim fingerprint sets across all datasets and architectures, demonstrating the integrity of Canary. This is because false claim fingerprints generated by malicious victims exhibit similar logits in the target and independent models, which is contrary to normal fingerprint generation objective. Thus, the verifier efficiently detects false claim fingerprints using posterior probabilities from its shadow independent model associated with false claim fingerprints and its dataset.

D. Evaluation on Efficiency

In this subsection, we evaluate the effectiveness and time overhead of Canary under different fingerprint set sizes in response to Q3. Specifically, we set three fingerprint set sizes, i.e., n = 30, n = 100, and n = 200. We evaluate the FPR and FNR of Grove and Canary at different fingerprint set sizes on the Citeseer dataset. In addition, we evaluate the time overhead of Grove and Canary (both are third-party model ownership verification methods, including the preparation process and the verification process) when parallel-processing different fingerprint set sizes. The preparation process starts when the victim initiates an ownership verification request and ends when suspicious model verification begins. The verification process starts then and ends upon obtaining the ownership result. Finally, we evaluate the time overhead of each part of Canary when the number of sampled subgraphs and fingerprints are 200, including the shadow model construction time T_{shadow} and fingerprint generation time T_{gen}, the fingerprint validation time T_{val}, the discriminator construction time $T_{\mathcal{B}}$ and the verification time T_{ver}.

Effectiveness at Different Fingerprint Set Sizes: As shown in Table VI, Canary can achieve a near-zero FPR and FNR with a small fingerprint set sizes, i.e., n=30. In contrast, when n=30, Grove achieves a 14.67% higher FNR on EmbYes, a 3.33% higher FNR on FTLL, and a 39.67% higher FNR on WP0.3 than Canary. Grove uses raw data as query input, which may contain nodes where the target model and surrogate models make different predictions, leading to significant differences in node embeddings and resulting in the misjudgments of Grove. Unlike Grove, for each fingerprint generated by Canary, it ensures that the predictions of the target model and the shadow surrogate model are the same, while the shadow independent model has different predictions, thereby reducing the misjudgments.

Time Overhead at Different Fingerprint Set Sizes: As shown in Table VII, during the preparation process, the time overhead of Canary is, on average, merely 0.67 seconds greater than that of Grove. Moreover, in the verification process, the time overhead of Canary is nearly identical to that of Grove. Even under the serial setting of Table VIII, Canary can complete ownership verification in about 1 seconds. Grove is a gray-box method, where the verifier directly uses the node embeddings to train the discriminator, resulting in less time overhead during the preparation phase. To protect the privacy of the target model, Canary involves the victim in generating a black-box fingerprint, thereby introducing additional time overhead. Considering the effectiveness of Canary demonstrated in Table VI, the time overhead is acceptable.

Time Overhead for Each Part of Canary: The time overhead results are shown in Table VIII, which demonstrates the efficiency of Canary in verifying model ownership. The model training time T_{shadow} is correlated with the dataset size (as shown in Table II) and the model architectures. For instance, Coauthor requires more time due to its large node and edge sizes. Moreover, the GAT architecture models require extended processing time because of their complex attention mechanisms, resulting in longer times for fingerprint generation T_{gen}, fingerprint validation T_{val}, and model ownership verification T_{ver} in comparison to GraphSAGE and GIN architectures. The training of the discriminator ($T_{\mathcal{B}}$) in Canary requires approximately 0.5 seconds. The verification process (T_{ver}) using the GAT architecture takes about 1 s to complete, while GraphSAGE and GIN require approximately 0.3 seconds.

E. Adversarial Robustness

In this subsection, we evaluate the robustness of Canary against five types of evasion detection methods by adversaries in response to Q4. Specifically, we evaluate the FNR of Canary and the average model accuracy of the surrogate models after applying these methods.

Post-processing Methods: We consider three post-processing methods that have been widely studied in existing verification methods [15], such as fine-tuning (e.g., Post-FTLL), retraining (e.g., Post-RTLL), and pruning (e.g., Post-WP0.3). Additionally, we investigate two potential post-processing methods where adversaries modify the posterior probabilities through DP and Top-K. The details are as follows.

- *Post-FTLL.* Execute FTLL again on the surrogate model.
- *Post-RTLL.* Execute RTLL again on the surrogate model.
- *Post-WP0.3.* Execute WP0.3 again on the surrogate model.
- *Differential Privacy.* Adversaries utilize (ϵ, δ)-DP [50] by adding Gaussian noise to the posterior probability under the privacy budget ϵ. In this subsection, we set $\epsilon = 5$ and $\epsilon = 10$, i.e., DP-5 and DP-10.
- *Top-k.* Adversaries hide the posterior probability and release only the probability vectors containing the classes with the top-k posterior probabilities. To mitigate this method, we assume that the classes without posterior probabilities have the same probability (i.e., $\frac{1-\sum(P_{\text{top-k}})}{N-k}$, where $P_{\text{top-k}}$ represents the top-k posterior probabilities and N

TABLE VI
THE EFFECTIVENESS OF GROVE AND CANARY AT DIFFERENT NUMBERS OF FINGERPRINTS ON THE CITESEER DATASET

Methods	FPR (%)			FNR (%)								
				EmbYes			FTLL			WP0.3		
	n=30	n=100	n=200	**n=30**	n=100	n=200	**n=30**	n=100	n=200	**n=30**	n=100	n=200
Grove [15]	8.42	7.19	6.81	14.67	3.67	0.00	3.33	0.00	0.00	39.67	33.33	33.33
Canary	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	1.67	**0.00**	**0.00**	0.67	**0.00**	**0.00**

TABLE VII
THE TIME OVERHEAD OF GROVE AND CANARY WITH DIFFERENT NUMBERS OF FINGERPRINTS ON THE CITESEER DATASET

Methods	Time Overhead in Preparation Process (s)			Time Overhead in Verification Process (s)		
	n=30	n=100	n=200	n=30	n=100	n=200
Grove [15]	9.63 (±0.75)	9.79 (±0.78)	9.93 (±0.76)	0.04 (±0.01)	0.04 (±0.00)	0.04 (±0.01)
Canary	10.10 (±0.61)	10.39 (±0.88)	10.86 (±0.72)	0.04 (±0.00)	0.04 (±0.00)	0.04 (±0.00)

TABLE VIII
THE TIME OVERHEAD OF CANARY

Models	Datasets	T_{shadow} (s)	T_{gen} (s)	T_{val} (s)	$T_{\mathcal{B}}$ (s)	T_{ver} (s)
GraphSAGE	Citeseer	9.41 (±0.72)	26.48 (±0.27)	0.56 (±0.05)	0.53 (±0.05)	0.32 (±0.01)
	Pubmed	47.77 (±0.98)	25.41 (±0.37)	0.56 (±0.06)	0.52 (±0.04)	0.32 (±0.01)
	DBLP	46.32 (±0.72)	28.08 (±0.61)	0.53 (±0.04)	0.56 (±0.11)	0.32 (±0.00)
	Coauthor	58.26 (±1.52)	28.42 (±0.54)	0.56 (±0.05)	0.54 (±0.16)	0.32 (±0.01)
	ACM	8.80 (±0.72)	26.64 (±0.39)	0.54 (±0.06)	0.55 (±0.06)	0.32 (±0.01)
	Amazon	53.07 (±0.74)	29.71 (±0.17)	0.57 (±0.05)	0.52 (±0.13)	0.33 (±0.01)
GAT	Citeseer	17.93 (±0.80)	76.73 (±0.47)	1.11 (±0.14)	0.56 (±0.13)	1.12 (±0.05)
	Pubmed	89.43 (±1.16)	78.27 (±0.73)	1.43 (±0.11)	0.56 (±0.08)	1.12 (±0.01)
	DBLP	85.29 (±0.74)	92.04 (±0.75)	1.66 (±0.41)	0.57 (±0.10)	1.12 (±0.04)
	Coauthor	123.27 (±0.77)	93.26 (±3.53)	1.29 (±0.05)	0.54 (±0.16)	1.13 (±0.04)
	ACM	15.44 (±0.79)	91.06 (±0.31)	1.43 (±0.12)	0.53 (±0.07)	1.11 (±0.01)
	Amazon	82.70 (±0.82)	89.76 (±0.25)	1.63 (±0.27)	0.55 (±0.19)	1.11 (±0.04)
GIN	Citeseer	7.93 (±0.58)	25.25 (±0.34)	0.56 (±0.07)	0.52 (±0.11)	0.33 (±0.04)
	Pubmed	30.48 (±1.02)	26.02 (±0.46)	0.52 (±0.12)	0.53 (±0.07)	0.33 (±0.01)
	DBLP	27.95 (±0.75)	28.75 (±0.67)	0.56 (±0.04)	0.56 (±0.09)	0.33 (±0.02)
	Coauthor	32.51 (±0.76)	27.77 (±0.94)	0.56 (±0.09)	0.59 (±0.12)	0.33 (±0.01)
	ACM	6.22 (±0.81)	26.84 (±0.58)	0.55 (±0.08)	0.56 (±0.12)	0.33 (±0.01)
	Amazon	17.00 (±0.82)	29.42 (±0.86)	0.56 (±0.11)	0.65 (±0.13)	0.34 (±0.01)

TABLE IX
THE FNR OF CANARY AND THE AVERAGE ACCURACY OF SURROGATE MODELS UNDER POST-PROCESSING METHODS

Methods	EmbYes		FTAL		RTAL		WP0.3	
	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$
Post-FTLL	2.00 %	-1.89 %	0.00	-0.18 %	0.00	-1.13 %	0.00	0.88 %
Post-RTLL	8.00 %	0.86 %	0.00	-0.49 %	0.00	-1.62 %	6.67 %	-0.44 %
Post-WP0.3	1.67 %	0.17 %	33.33 %	-30.3 %	33.33 %	-15.03 %	49.33 %	-33.59 %
DP-5	38.33 %	-20.98 %	16.00 %	-11.77 %	20.33 %	-13.08 %	19.33 %	-12.76 %
DP-10	0.00	-12.77 %	3.00 %	-2.09 %	0.00	-5.65 %	0.67 %	-6.02 %
Top-2	2.00 %	0.00	0.00	0.00	0.67 %	0.00	0.00	0.00
Top-5	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

represents the total number of classes). Here, we set $k = 2$ and $k = 5$, i.e., Top-2 and Top-5.

The results shown in Table IX demonstrate that Canary is robust against these post-processing methods. Compared with no evasion methods, Canary only increases a 0%–8% higher FNR for Post-FTLL, Post-RTLL. Although Canary increases a 1.67%–49.33% FNR for Post-WP0.3, surrogate models reduce an 0.17%–33.59% accuracy simultaneously. When adversaries attempt to evade detection by modifying parameters or posterior probabilities, they inevitably reduce the model accuracy. Consequently, Canary can effectively identify surrogate models that maintain certain accuracy.

Adaptive Attacks: Here, we evaluate the robustness of Canary in defending against two more powerful adaptive attacks, where adversaries have knowledge of the fingerprint optimization objectives of Canary and avoid replicating decision boundaries close to the target model to evade detection. *1) Adapt-bound:* The adversary queries the target model to identify input samples

TABLE X
THE FNR OF CANARY AND THE AVERAGE ACCURACY OF SURROGATE MODELS UNDER ADAPTIVE ATTACKS

Methods	EmbYes		FTAL		RTAL		WP0.3	
	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$	Δ_{FNR}	Model $\Delta_{\text{Acc.}}$
Adapt-bound-2%	36.67 %	-19.17 %	0.77 %	-0.39 %	4.31 %	-0.48 %	0.00	-0.81 %
Adapt-bound-5%	58.67 %	-29.28 %	17.67 %	-6.71 %	61.34 %	31.51 %	23.33 %	-7.36 %
Adapt-nf-2%	38.00 %	-15.83 %	-2.33 %	-0.46 %	4.00 %	-0.29 %	0.00	-0.64 %
Adapt-nf-5%	64.67 %	-34.80 %	-4.00 %	-0.79 %	61.34 %	19.89 %	0.33 %	-0.68 %

with low $\mathcal{L}_{\text{bound}}$ in (5), which reside near decision boundaries of the target model. These samples are then leveraged in an inverse optimization process to significantly diverge the decision boundary of the surrogate model from that of the target model. *2) Adapt-natural fingerprints (nf):* The adversary queries the target model, an independent model, and its surrogate model, filtering for samples where the predictions of the target model differ from those of the independent model but match those of the surrogate model. These samples represent natural fingerprints and are used to reverse optimize the surrogate model.

We conduct adaptive attacks, i.e., Adapt-bound-2%, Adapt-bound-5%, Adapt-nf-2% and Adapt-nf-5%, on the Citeseer dataset, where the adversaries query the target model using their datasets and utilize the 2% and 5% nodes with the lowest $\mathcal{L}_{\text{bound}}$ or with the aforementioned prediction differences to reversely optimize their surrogate models. The results in Table X show that Canary achieves comparable verification effectiveness unless model stealing fails. Canary only increases a 0%–4.34% higher FNR for Adapt-bound-2% and Adapt-nf-2% than that with no evasion methods, while the surrogate models reduce a 0.29%–0.64% accuracy. Compared with no evasion methods, Canary increases a 0.33%–64.67% higher FNR for Adapt-bound-5% and Adapt-nf-5% as the surrogate models reduce a 0.68%–34.8% accuracy.

F. Sensitivity Analysis

In this subsection, we perform the sensitivity analysis of Canary with respect to the loss functions, the distance metrics, the assumptions regarding data scale and distribution, the parameters λ_1, λ_2, the subgraph hop number (i.e., subgraph size), and shadow surrogate model types in response to Q5.

Ablation Experiment on Loss Function: We conduct ablation experiments of the loss function of fingerprints on Citeseer datasets to prove their necessity. Specifically, we respectively remove the $\mathcal{L}_{\text{corr}}$, $\mathcal{L}_{\text{diff}}$, $\mathcal{L}_{\text{bound}}$, and $\mathcal{L}_{\text{final}}$ in Canary to generate fingerprints. Then the generated fingerprints are used to test the FPR of independent models and the FNR of surrogate models.

The FPR and FNR are recorded in Table XI and demonstrate the effectiveness of the loss functions. Specifically, compared with the unablated canary, the ablated Canary increases a 1.67%–25.33% higher FPR and a 3%–36% higher FNR. After the ablation of $\mathcal{L}_{\text{corr}}$, and $\mathcal{L}_{\text{diff}}$, the Canary will generate fingerprints that cause different predictions of the target model and surrogate models, or for fingerprints that cause the same predictions from the target model and independent models, resulting in misjudgments by the Canary. The ablation of $\mathcal{L}_{\text{bound}}$ will cause the fingerprints to contain less unique decision knowledge from the target model, thus reducing the effectiveness of the Canary. Here, the fingerprints obtained by ablating $\mathcal{L}_{\text{final}}$ can be regarded as the inherent fingerprints derived from the real dataset. Such fingerprints exhibit limited transferability across different model stealing attacks.

TABLE XI
EFFECTIVENESS ON CITESEER AFTER LOSS ABLATION

Methods	FPR (%)	FNR (%)			
		EmbYes	FTAL	RTAL	WP0.3
w/o $\mathcal{L}_{\text{corr}}$	1.67	24.33	8.33	20.33	36.00
w/o $\mathcal{L}_{\text{diff}}$	25.33	22.67	33.33	33.33	33.33
w/o $\mathcal{L}_{\text{bound}}$	3.67	16.00	11.67	24.33	33.33
w/o $\mathcal{L}_{\text{final}}$	4.50	5.33	18.50	18.33	16.67
Canary	0.00	0.00	5.33	4.33	0.00

TABLE XII
EFFECTIVENESS ON ACM AFTER DISTANCE METRIC ABLATION

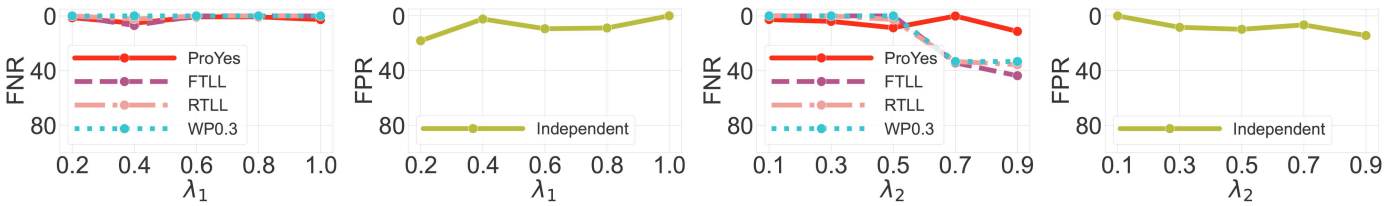
Methods	FPR (%)	FNR (%)			
		EmbYes	FTAL	RTAL	WP0.3
w/o Cos	35.33	0.00	0.67	0.00	0.00
w/o ρ	36.67	0.00	0.67	0.00	0.00
w/o ℓ_∞	27.22	0.00	0.67	0.00	0.00
w/o ℓ_2	35.67	0.00	0.67	0.00	0.00
w/o KL	21.78	0.00	0.67	0.00	19.67
w/o JS	39.78	0.00	0.67	0.00	0.00
Canary	17.11	0.00	1.00	0.00	3.00

Ablation Experiment on Distance Metrics: We conduct ablation experiments on the distance metrics for discriminator training on the ACM dataset (with the lowest F1-neg) to demonstrate their necessity for distinguishing between independent models and surrogate models. Specifically, we respectively remove the six distance metrics as described in Section V-C to train the discriminator and verify ownership.

The FPR and FNR recorded in Table XII demonstrate the necessity of six distance metrics. Compared with the unablated canary, the ablated Canary increases a 4.67%–22.67% higher FPR, while the FNR remains almost unaffected. The reason is that, compared to independent models, surrogate models possess decision boundaries that are more similar to those of the target model. Recognizing the similarities between the target and surrogate models is relatively straightforward; however, identifying the differences between the target and independent models is more challenging. Therefore, employing more comprehensive distance metrics is more helpful in distinguishing between surrogate and independent models.

TABLE XIII
THE IMPACT OF DATA ASSUMPTIONS ON THE EFFECTIVENESS OF CANARY ON THE PUBMED DATASET

Verifier Data	Adversary Data	FPR (%)	FNR (%)							
			Extraction		Fine-tuning		Retraining		Pruning	
			EmbYes	ProNo	FTLL	FTAL	RTLL	RTAL	WP0.3	WP0.9
20% (i.i.d)	20% (i.i.d)	18.67	0.00	3.00	0.00	0.00	1.00	0.00	0.00	0.00
20% (i.i.d)	5% (non-i.i.d)	18.67	0.00	1.33	0.00	0.00	2.00	0.00	0.00	0.00
5% (non-i.i.d)	20% (i.i.d)	17.33	0.00	0.33	0.00	0.00	0.00	0.00	0.00	33.33
5% (non-i.i.d)	5% (non-i.i.d)	17.33	1.00	2.33	0.00	0.00	0.33	0.00	0.00	0.00



(a) FNR (λ_1) (b) FPR (λ_1) (c) FNR (λ_2) (d) FPR (λ_2)

Fig. 4. The impact of λ_1 and λ_2 on the effectiveness of Canary on the Citeseer dataset.

The Impact of Relaxing Data Assumptions: We relax the assumptions that both the verifier and the adversary operate on data drawn from a distribution similar to that used by the target model. Concurrently, we also relax the assumption that the victim, verifier, and adversaries possess data of equivalent scale in the experimental setup of Section VI-A. Specifically, for the largest Pubmed dataset containing three classes, we change datasets of the verifier and adversaries by adjusting the proportions of the nodes across these classes from approximately {20%, 40%, 40%} (i.i.d) to {25%, 25%, 50%} (non-i.i.d), while also reducing their proportion in the original dataset from 20% to 5%.

As shown in Table XIII, we observe that relaxing the assumptions of the dataset distribution and scale does not significantly affect Canary's effectiveness. After relaxing only the data assumptions of the verifier, the FPR changes by only 1.34%. Notably, when the data assumptions of the verifier are relaxed, the FNR for for WP0.9 increases to 33.33%, which is primarily due to the average accuracy of the surrogate models being only 56.86%. This relaxation of the data assumptions of the verifier results in a decline in the performance of the discriminator, thereby making it more difficult to identify these unusable surrogate models. When the data assumptions of adversaries are relaxed, the surrogate models can be fine-tuned with reduced datasets, achieving an average accuracy of 86.08%, while Canary maintains a 0% FNR.

The Impact of the Difference Weight λ_1: With λ_2 fixed at 0.1, we record the FNR and FPR on the Citeseer dataset, while varying λ_1 among the values 1.0, 0.8, 0.6, 0.4, and 0.2. As shown in Fig. 4(a), the effectiveness of Canary in identifying surrogate models is not significantly impacted by variations in λ_1. As λ_1 decreases, the effectiveness of Canary in identifying independent models gradually declines as shown in Fig. 4(b). The reduction in λ_1 causes the posteriors of the fingerprints for the target model and independent models to become increasingly similar, thereby diminishing the effectiveness of identifying independent models.

The Impact of Boundary Weight λ_2: With λ_1 fixed at 1.0, we record the FNR and the FPR while varying λ_2 among the values 0.1, 0.3, 0.5, 0.7, and 0.9. Increasing the λ_2 will reduce the effectiveness of the Canary. As illustrated in Fig. 4(c) and 4(d), when λ_2 exceeds 0.5, the FNR and FPR of Canary increase. The loss component λ_2 increases the information entropy that reflects the unique decision-making knowledge of the target model within the fingerprint. While smaller values of λ_2 can enhance fingerprint performance, larger values may influence the primary loss components, i.e., $\mathcal{L}_{\text{corr}}$ and $\mathcal{L}_{\text{diff}}$, potentially compromising the effectiveness.

The Impact of the Subgraph Size: We fix the optimization epoch for subgraphs at 40 and record the FNR, the FPR, and the time required to generate fingerprints (T_{gen}) and verify (T_{ver}) a single fingerprint while varying the hop number n among the values 2, 5, and 10. As shown in Table XIV, the effectiveness of Canary decreases with an increasing hop number n (i.e., subgraph size), although the impact on fingerprint generation and verification times remains minimal. This reduction in effectiveness is attributed to the complexity introduced by the increased hop number n, which necessitates more iterations for successful optimization of fingerprint generation. Within a limited number of iterations, the fingerprints optimized with simpler subgraph structures are more effective. Additionally, the fingerprint generation time and verification time are primarily influenced by the model architectures. The GAT architecture, due to its complex attention mechanism, has a higher fingerprint generation time and verification time compared to GraphSAGE and GIN.

The Impact of Different Shadow Surrogate Models: We evaluate scenarios in which the shadow surrogate model is constructed using a range of model stealing attacks with varying strengths, i.e., FTLL, RTLL, and WP0.3. The results validate the core

TABLE XIV
EFFECTIVENESS AND EFFICIENCY OF DIFFERENT HOP NUMBERS ON THE CITESEER DATASET

Hops	Effectiveness (%)					Efficiency (s)					
	FPR	FNR				GraphSAGE		GAT		GIN	
		EmbYes	FTAL	RTAL	WP0.3	T_{gen}	T_{ver}	T_{gen}	T_{ver}	T_{gen}	T_{ver}
10-hop	0.00	0.33	23.67	8.67	10.33	0.31	0.04	0.51	0.06	0.30	0.04
5-hop	0.00	0.33	16.33	7.33	8.67	0.27	0.03	0.48	0.04	0.26	0.03
2-hop	0.00	0.00	5.33	4.33	0.00	0.26	0.03	0.47	0.04	0.26	0.03

TABLE XV
THE IMPACT OF THE SHADOW SURROGATE MODEL CONSTRUCTION METHOD ON THE EFFECTIVENESS OF CANARY

Methods	FPR (%)	FNR (%)										F1-neg (%)
		Extraction				Fine-tuning		Retraining		Pruning		
		ProYes	EmbYes	ProNo	EmbNo	FTLL	FTAL	RTLL	RTAL	WP0.3	WP0.9	
Canary with ProYes	0.00	0.00	0.00	15.67	11.33	0.00	5.33	0.00	4.33	0.00	0.00	94.24
Canary with FTLL	0.00	29.00	63.33	41.00	35.33	0.00	0.00	0.00	0.00	0.00	3.67	77.69
Canary with RTLL	0.00	10.00	10.33	31.67	32.00	0.00	0.00	0.00	0.00	0.00	0.00	87.72
Canary with WP0.3	0.00	61.33	67.33	91.67	70.67	1.00	2.00	0.00	2.67	0.00	6.67	66.42

design of Canary that a shadow surrogate model built under a weaker attack can effectively detect stronger ones.

Specifically, we evaluate the FPR, FNR, and F1-neg on the Citeseer dataset under different shadow surrogate models constructed using stronger white box model stealing attacks, i.e., FTLL, RTLL, and WP0.3, and compare these results with those obtained using the weaker black box attack ProYes.

As shown in Table XV, when Canary employs shadow surrogate models constructed by stronger white-box model stealing attacks (i.e., FTLL, RTLL, and WP0.3), the FNR for detecting black-box and gray-box attacks (i.e., ProYes, EmbYes, ProNo, and EmbNo) increases by 10%–76%. This performance reduction arises because stronger white-box attacks produce shadow surrogate models whose decision boundaries closely align with that of the target model, leaving Canary unable to learn features of surrogate models that deviate more substantially from the target model. In contrast, weaker attacks induce larger decision boundary deviations, which preserve a richer set of behavioral features that remain effective against stronger model extraction attacks.

VII. DISCUSSION

Generalizability to Other GNN Tasks: In this paper, we focus on the node classification task, but Canary can be generalized to other GNN tasks. This is because the fingerprint validation and the construction of the ownership discriminator in Canary are based on the posterior probabilities of fingerprints, which can also be applied to other GNN classification tasks that output posterior probabilities, such as graph classification [51] and edge [52] classification. Considering that graph classification and edge classification necessitate the aggregation of node features into graph-level and edge-level representations, Canary can sample larger subgraphs to generate fingerprints that maintain their effectiveness in these GNN tasks. In addition, the aggregation operations of graph-level features and edge-level features are typically differentiable, so the backpropagation-based fingerprint feature optimization process is still applicable.

Effectiveness on Other GNN Architectures: Canary can maintain its effectiveness on architectures such as the Graph Convolutional Network (GCN) [40], the Simple Graph Convolution (SGC) [53], and the Graph Transformer (GT) [54]. The ownership verification of Canary requires the gradients from the backpropagation and the posterior probabilities of models. For the gradients, GCN aggregates node features through convolutional operations and can accurately obtain gradients during backpropagation using the convolutional path. SGC simplifies the non-linear activation functions of GCN, and its linear structure enables stable gradient backpropagation. GT utilizes the self-attention mechanism to capture global information. Although computationally complex, it can generate gradients by comprehensively considering node relationships.

Dynamic Threshold: We discuss an extreme scenario that might affect the effectiveness of a fixed 50% threshold and propose a dynamic threshold strategy to address it.

The optimization objective of Canary is to generate fingerprints that yield consistent predictions between the target model and surrogate models but inconsistent predictions on independent models. However, in scenarios such as machine learning competitions or domains with a single dominant public dataset, developers often use the same data and mainstream model architectures. This high degree of homogeneity in both datasets and model structures significantly reduces the divergence between the decision boundaries of the independent and target models. Consequently, the FMR on independent models may exceed 50%, leading to misclassification.

To address this issue, we propose a dynamic threshold based on hypothesis testing. The core idea is that even in the aforementioned scenario, a statistically significant difference persists between the fingerprint responses of independent models and those of surrogate models. We assume that the verifier can access or train a set of independent models as references and obtain a set of FMR on them. At a confidence level of 1-α (with α representing the significance level), a one-tailed T-test [55] can be employed to determine τ, the upper limit of the confidence

TABLE XVI
THE IMPACT OF THE THRESHOLD τ ON THE EFFECTIVENESS OF CANARY

Datasets	Methods	FPR (%)	FNR (%)										F1-neg (%)
			Extraction				Fine-tuning		Retraining		Pruning		
			ProYes	EmbYes	ProNo	EmbNo	FTLL	FTAL	RTLL	RTAL	WP0.3	WP0.9	
Citeseer	Fixed	0.00	0.00	0.00	15.67	11.33	0.00	5.33	0.00	4.33	0.00	0.00	94.24
	Dynamic	13.11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	92.96
Pubmed	Fixed	18.67	1.33	0.00	3.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	88.83
	Dynamic	14.45	0.00	0.00	1.67	0.00	0.00	0.33	0.00	0.00	0.00	0.00	91.88
DBLP	Fixed	0.00	17.00	0.00	18.67	0.00	0.00	0.00	0.00	1.33	0.00	0.00	94.19
	Dynamic	9.78	6.66	0.00	9.67	0.00	0.00	0.00	0.00	0.00	0.00	0.00	92.22
Coauthor	Fixed	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	33.33	33.33	90.00
	Dynamic	7.78	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.33	1.00	95.57
ACM	Fixed	17.11	5.00	0.00	10.67	2.00	0.00	1.00	0.00	0.00	3.00	0.00	87.20
	Dynamic	10.66	5.67	0.00	12.67	4.33	0.00	1.00	0.00	0.00	5.33	0.00	89.78
Amazon	Fixed	19.78	1.67	0.00	5.00	0.00	0.00	0.00	0.00	3.33	0.00	0.00	87.41
	Dynamic	15.78	2.00	0.67	5.67	2.67	2.00	2.00	0.00	4.33	0.00	0.00	89.39

interval. This threshold approximates the upper bound of the FMR distribution for independent models.

Our experiments demonstrate the effectiveness of the dynamic threshold. Specifically, we allow the verifier to query 30 independent models to collect a set of FMRs and compute the threshold τ. As shown in Table XVI, on the Pubmed, ACM, and Amazon datasets, the FPRs of the fixed-threshold Canary are 18.67%, 17.11%, and 19.78%, respectively. This indicates that for these datasets, the FMRs of some independent models exceed 50%. In contrast, the dynamic-threshold Canary adaptively adjusts the decision boundary according to the statistical distribution of the independent models, thereby reducing the FPR by 4.00%–6.45% and increasing the F1-neg by 1.98%–3.05%. Although these results are obtained under the experiment settings in Section VI rather than the aforementioned extreme scenario, they demonstrate that when the FMR of independent models exceeds 50%, the dynamic threshold strategy exhibits stronger verification effectiveness.

VIII. Related Works

Watermark-based Methods: Watermark-based methods directly embed watermarks into the model parameters, such as fully connected layers [56] or activations [57] during training, and extract this information during ownership verification. Alternatively, watermarks can be embedded in parameters using triggers, such as image triggers with identifiers [58], a random graph trigger [19] or node feature triggers [59], and verified by inputting the triggers and checking the model's output. However, black-box watermarking is vulnerable to removal attacks [60], while white-box methods [20] risk privacy leakage to third-parties.

Fingerprint-based Methods: Fingerprint-based model ownership verification methods preserve model performance without modifying the model. Recent works validate ownership by comparing intrinsic behavioral consistency between target and suspicious models, such as comparing the neuron activation distance similarity [21], analyzing the GNN node embedding similarity [15], posterior probability similarity [16] and examining the output consistency against adversarial samples [17], [18], [22]. However, GNN fingerprint works either require the verifier to have gray-box access to the target model [15], [18] or rely on the same model stealing attack as the adversary to extract fingerprints [16], [17]. Canary generates fingerprints by maximizing the difference between surrogate and independent models, allowing the verifier to effectively identify other strgonger model stealing attacks in a black-box scenario by constructing the shadow model using the weaker attack.

IX. Conclusion

In this paper, we proposed Canary, a fingerprint-based black-box verification protocol for GNN ownership. Canary generated subgraph inputs as decision boundary fingerprints by maximizing the logits distance between the target model and independent models, while minimizing the distance from surrogate models. These fingerprints effectively revealed the posterior probability discrepancy between independent models and surrogate models under the fingerprint queries. By conducting thorough experiments across six datasets and three model architectures, Canary achieved better performance than the fingerprint-based gray-box and black-box method and robustly resists false claims. In future work, we will further improve the accuracy and robustness of Canary.

References

[1] H. Du et al., "Fine-grained and class-incremental malicious account detection in ethereum via dynamic graph learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 10130–10145, 2025.
[2] Z. Che et al., "Across-platform detection of malicious cryptocurrency accounts via interaction feature learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 4783–4798, 2025.
[3] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 2367–2380, 2021.
[4] F. Huang, P. Yi, J. Wang, M. Li, J. Peng, and X. Xiong, "A dynamical spatial-temporal graph neural network for traffic demand prediction," *Inf. Sci.*, vol. 594, pp. 286–304, 2022.
[5] S. Adeshina, "Detecting fraud in heterogeneous networks using Amazon SageMaker and deep graph library," 2020. [Online]. Available: https://aws.amazon.com/blogs/machine-learning/detecting-fraud-in-heterogeneous-networks-using-amazon-sagemaker-and-deep-graph-library/
[6] B. Balakreshnan, "Graph neural network in azure machine learning–Classification," 2021. [Online]. Available: https://balabala76.medium.com/graph-neural-network-in-azure-machine-learning-classification-df7978d7bd45
[7] Y. Shen, X. He, Y. Han, and Y. Zhang, "Model stealing attacks against inductive graph neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1175–1192.

[8] B. Wu, X. Yang, S. Pan, and X. Yuan, "Model extraction attacks on graph neural networks: Taxonomy and realisation," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2022, pp. 337–350.

[9] N. Pittaras, F. Markatopoulou, V. Mezaris, and I. Patras, "Comparison of fine-tuning and extension strategies for deep convolutional neural networks," in *Proc. 23rd Int. Conf. MultiMedia Model.*, Reykjavik, Iceland, Springer, 2017, pp. 102–114.

[10] Y. Adi, C. Baum, M. Cisse, B. Pinkas, and J. Keshet, "Turning your weakness into a strength: Watermarking deep neural networks by backdooring," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1615–1631.

[11] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *Proc. 7th Int. Conf. Learn. Representations*, 2019.

[12] M. Shen, H. Lu, A. Gu, Q. Li, K. Xu, and L. Zhu, "PriGraph: Defending against inference attacks on graph neural networks via policy-based adversarial perturbations," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 1, pp. 890–904, Jan./Feb. 2026.

[13] M. Shen, C. Li, Q. Li, H. Lu, L. Zhu, and K. Xu, "Transferability of white-box perturbations: Query-Efficient adversarial attacks against commercial DNN services," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 2991–3008.

[14] M. Shen et al., "Casper: A causality-inspired defense with confounder against label inference attacks in vertical split federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 21, pp. 1050–1064, 2026.

[15] A. Waheed, V. Duddu, and N. Asokan, "GrOVe: Ownership verification of graph neural networks using embeddings," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 2460–2477.

[16] R. Zhou, K. Yang, X. Wang, W. H. Wang, and J. Xu, "Revisiting black-box ownership verification for graph neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 210–210.

[17] X. You, Y. Jiang, J. Xu, M. Zhang, and M. Yang, "GNNFingers: A fingerprinting framework for verifying ownerships of graph neural networks," in *Proc. ACM Web Conf.*, 2024, pp. 652–663.

[18] J. Jia, R. Li, C. Wu, S. Ma, L. Wang, and R. H. Deng, "SIGFinger: A subtle and interactive GNN fingerprinting scheme via spatial structure inference perturbation," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 3629–3646, Jul./Aug. 2025.

[19] X. Zhao, H. Wu, and X. Zhang, "Watermarking graph neural networks by random graphs," in *Proc. 9th Int. Symp. Digit. Forensics Secur.*, 2021, pp. 1–6.

[20] T. Krauß, J. Stang, and A. Dmitrienko, "ClearStamp: A Human-Visible and robust Model-Ownership proof based on transposed model training," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 5269–5286.

[21] J. Chen et al., "Copy, right? A testing framework for copyright protection of deep learning models," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 824–841.

[22] Y. Chen, C. Shen, C. Wang, and Y. Zhang, "Teacher model fingerprinting attacks against transfer learning," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3593–3610.

[23] S. Yang et al., "Financial risk analysis for SMEs with graph-based supply chain mining," in *Proc. 29th Int. Conf. Int. Joint Conf. Artif. Intell.*, 2021, pp. 4661–4667.

[24] C. L. Giles, K. D. Bollacker, and S. Lawrence, "CiteSeer: An automatic citation indexing system," in *Proc. 3rd ACM Conf. Digit. Libraries*, 1998, pp. 89–98.

[25] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," 2018, *arXiv:1811.05868*.

[26] J. McAuley, C. Targett, Q. Shi, and A. Van Den Hengel, "Image-based recommendations on styles and substitutes," in *Proc. 38th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2015, pp. 43–52.

[27] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Mag.*, vol. 29, no. 3, pp. 93–93, 2008.

[28] S. Pan, J. Wu, X. Zhu, C. Zhang, and Y. Wang, "Tri-party deep network representation," *Network*, vol. 11, no. 9, 2016, Art. no. 12.

[29] X. Wang et al., "Heterogeneous graph attention network," in *Proc. World Wide Web Conf.*, 2019, pp. 2022–2032.

[30] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.

[31] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. 6th Int. Conf. Learn. Representations*, 2018.

[32] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. Int. Conf. Learn. Representations*, 2019.

[33] M. Song, Z. Su, X. Qu, J. Zhou, and Y. Cheng, "PRMBench: A fine-grained and challenging benchmark for process-level reward models," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 25299–25346.

[34] Y. Liu et al., "Provenance of training without training data: Towards privacy-preserving DNN model ownership verification," in *Proc. ACM Web Conf.*, 2023, pp. 1980–1990.

[35] Y. Chen, L. Wu, and M. Zaki, "Iterative deep graph learning for graph neural networks: Better and robust node embeddings," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 19314–19326.

[36] M. Yan, C. W. Fletcher, and J. Torrellas, "Cache telepathy: Leveraging shared resource attacks to learn DNN architectures," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 2003–2020.

[37] D. DeFazio and A. Ramesh, "Adversarial model extraction on graph neural networks," 2019, *arXiv:1912.07721*.

[38] J. Liu, R. Zhang, S. Szyller, K. Ren, and N. Asokan, "False claims against model ownership resolution," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 6885–6902.

[39] Y. Chang, H. Jiang, C. Lin, X. Huang, and J. Weng, "Towards understanding and enhancing Secur. of proof-of-training for DNN model ownership verification," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 7233–7250.

[40] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[41] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 249–256.

[42] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1026–1034.

[43] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.

[44] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 1532–1543.

[45] K. Pearson, "Notes on the history of correlation," *Biometrika*, vol. 13, no. 1, pp. 25–45, 1920.

[46] R. Coghetto, "Chebyshev distance," *Formalized Math.*, vol. 24, no. 2, pp. 121–141, 2016.

[47] P.-E. Danielsson, "Euclidean distance mapping," *Comput. Graph. Image Process.*, vol. 14, no. 3, pp. 227–248, 1980.

[48] T. Van Erven and P. Harremos, "Rényi divergence and Kullback-Leibler divergence," *IEEE Trans. Inf. Theory*, vol. 60, no. 7, pp. 3797–3820, Jul. 2014.

[49] M. L. Menéndez, J. Pardo, L. Pardo, and M. Pardo, "The Jensen-Shannon divergence," *J. Franklin Inst.*, vol. 334, no. 2, pp. 307–318, 1997.

[50] C. Dwork et al., "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3/4, pp. 211–407, 2014.

[51] X. Han, Z. Jiang, N. Liu, and X. Hu, "G-Mixup: Graph data augmentation for graph classification," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 8230–8248.

[52] X. Cheng, Y. Wang, Y. Liu, Y. Zhao, C. C. Aggarwal, and T. Derr, "Edge classification on graphs: New directions in topological imbalance," in *Proc. 18th ACM Int. Conf. Web Search Data Mining*, 2025, pp. 392–400.

[53] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.

[54] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 14501–14515.

[55] C. A. Boneau, "The effects of violations of assumptions underlying the t test," *Psychol. Bull.*, vol. 57, no. 1, 1960, Art. no. 49.

[56] M. Kuribayashi, T. Tanaka, and N. Funabiki, "DeepWatermark: Embedding watermark into DNN model," in *Proc. Asia-Pacific Signal Inf. Process. Assoc. Annu. Summit Conf.*, 2020, pp. 1340–1346.

[57] B. Darvish Rouhani, H. Chen, and F. Koushanfar, "DeepSigns: An end-to-end watermarking framework for ownership protection of deep neural networks," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2019, pp. 485–497.

[58] J. Zhang et al., "Protecting intellectual property of deep neural networks with watermarking," in *Proc. Asia Conf. Comput. Commun. Secur.*, 2018, pp. 159–172.

[59] J. Xu, S. Koffas, O. Ersoy, and S. Picek, "Watermarking graph neural networks based on backdoor attacks," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy*, 2023, pp. 1179–1197.

[60] Y. Lu, W. Li, M. Zhang, X. Pan, and M. Yang, "Neural dehydration: Effective erasure of black-box watermarks from DNNs with limited data," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 675–689.